

INT102

内容完全来自于LearningMall的PPT。仅供学习和交流使用。任何机构或个人不得将其用于商业用途或未经授权的传播。如需引用或分享，请注明出处并保留此声明。

 GITHUB

REPOSITORY

 VISIT NOTES AUTHOR'S PAGE

CLICK ME

CPT102 **Data Structures and Algorithms** 数据结构和算法

算法基础与问题求解 Algorithmic Foundations And Problem Solving

■ [Week 1

Week 1 Lecture 1 + Week 1 Lecture 2 (Tutorial)

[0] 课程情况

教师信息

Teacher Name	Office Room	Email	Office Hour
Dr. Jia Wang (M. Leader)	SD537	jia.wang02@xjtlu.edu.cn	14:00-16:00, Tuesday
Dr. Yushi Li	SD561	yushi.li@xjtlu.edu.cn	14:00-16:00, Tuesday
Dr. Pengfei Fan	SD559	pengfei.fan@xjtlu.edu.cn	14:00-16:00, Tuesday

学分组成

评估项	占比	其他
Assessment 1	10%	week 6 - week 7
Assessment 2	10%	week 12 – week 13
Final Examination	80%	Date TBA.

总内容

Methodology/Problems	Asymptotic Idea 渐进思想	Brute Force 蛮力法 (暴力)	Divide & Conquer 分治法	Dynamic Programming (动态规划)	Greedy 贪心	Space/Time	Branch & Bound 分支界定法	Complexity Theory 复杂性理论
Efficiency 效率	Big-O							
Sorting 排序		Selection/Bubble/insertion	Mergesort		Count sorting			
Searching 搜索			Binary-searching					
String Matching 串匹配						Horspool algorithm		
Graph 图		DFS/BFS		Bellman-ford/Warshall/Assembly-line	MST/Dijkstra's/Shortest path		Traveling salesman, Job assignment	
Complexity 复杂性								P/NP

程序 = 数据结构 + 算法

[1] 伪代码简述

伪代码 (Pseudo)

```
1  p = 1 # 赋值
2  _____
3  for i = 1 to n do # 循环
4      p = p * x
5  _____
6  for i = 1 to n do # 循环 2
7      begin
8          statement
9      end
10 _____
11 output p # 输出
12 _____
13 if p < 0 then # if 语句
14     output -p
15 _____
16 while a > 10 do # while 语句
17     statement
18 _____
19 while condition do # while 语句2
20     begin
21         a = ask for a number # 输入数字
22     end
23 _____
24 repeat # repeat语句
25     statement
26 until a<0
27 _____
```

小练习

1. Find the product of all integers in the interval $[x, y]$, assuming that x and y are both integers

寻找连续区间 $[x, y]$ 的乘积。例如 $[4, 10] = 4 \times 5 \times \dots \times 10$

```
1  sum = 1
2  for i = x to y do
3      begin
4          sum *= i
5      end
6  # sum *= i 即 sum = sum * i; 同时begin和end可省去（无歧义）;
```

2. List all factors of a given positive integer x

列出所有 x 的因数

```
1 for i = 1 to x do
2     if x % i == 0 then
3         output i
```

能否把他们转为 `while` 和 `repeat` 呢？尝试一下吧。

为什么我们需要更好的算法？ 接下来是一个直观的例子。

假设计算机的速度是：5 次操作/s

现在有一个算法的速度如下：比较 n 个数，需要 n^2 次操作

因此，处理 5 个数据需要 $5 \times 5 = 25$ 次操作，也就是 5 s

假设把计算机速度提高100倍，也就是 500 次操作 / s

如果不优化算法，在相同时间 5 s内，只能完成 $\sqrt{5 \times 500} = 50$ 个数据。

可见，即便把计算机速度提高 100 倍，如果不改变算法，也只能处理比原来多 10 倍的数据。

[2] 多项式

涉及 n 的幂的函数（例如， n ， $n\log(n)$ ， n^2 ，称为**多项式函数**）与 n 作为指数的函数（例如， 2^n ，称为**指数函数**）之间存在巨大差异。简而言之就是 n 在上面还是下面差别很大。

在算法分析和时间复杂度中， $\log(n)$ 通常表示的是以 2 为底的对数，即 $\log_2(n)$ 。

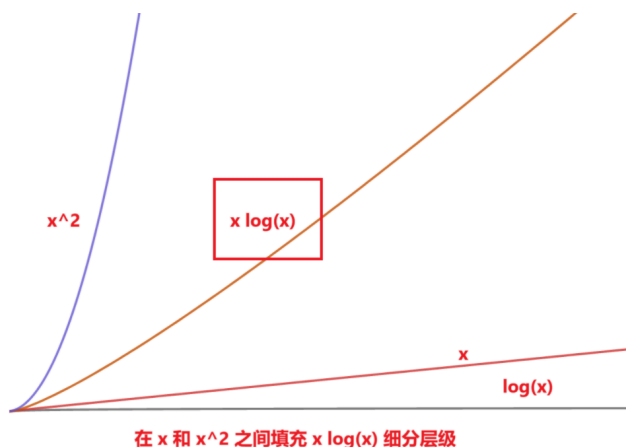
多项式函数 polynomial functions

指数函数 exponential functions

对于多项式函数而言，齐次幂的更具有可比性。例如 $f_3(n) = n^2 - 3^n + 6$ 和 $f_4(n) = 2n^2$

我们可以定义一个函数的**层级 (Hierarchy)**，每个函数的数量级都比其前一个函数要大。对于较大的数据量而言，层级小的函数所用次数少： c (常数) $< \log(n) < n < n^2 < \dots$ 。

我们可以通过在 n 和 n^2 之间插入 $n\log(n)$ ，在 n^2 和 n^3 之间插入 $n^2\log(n)$ 等等来进一步细化这个层级。



[3] 大O符号

现在，当我们有一个函数时，我们可以将这个函数归类到**层级**中的某个函数。

例如: $f(n) = 2n^3 + 5n^2 + 4n + 7 \rightarrow$ 归类到 n^3 因为主要受 n^3 的影响。

这个概念通过大 O 符号 (Big-O notation) 表示。

$$f(n) = O(g(n))$$

其中 $g(n) = f(n)$ 的最高次项(除常数)。例如 $2n^3 = O(n^3)$ 、 $n^3 + n^2 = O(n^3)$

左右两侧的函数被认为具有**相同数量级(Same order of magnitude)**。

正式定义

$f(n) = O(g(n))$: 存在一个常数 c 和 n_0 , 使得对于所有 $n > n_0$, $f(n) \leq c * g(n)$.

这个难以理解没关系，会用就可以。例如函数 $f(n) = 2n^3$ ，我们可以找到一个常数 c 比如 $c = 2 + 1 = 3$ ，和一个 n_0 比如 1，使得对于所有 $n > 1$ ， $2n^3$ 始终比 $3n^3$ 小。这时候，我们就可以说 $f(n) = O(n^3)$ 。

如果说不是那么明显的话，例如函数是 $f(n) = n^6 + n^5 + \dots + n$ ，我们就可以找到一个常数，例如 $c = 1 + 1 = 2$ ，然后再找一个非常大的 $n = 1,000,000$ 。使得对于所有 $n > 1,000,000$ 总有 $f(n) = n^6 + \dots + n < 2n^6$

```
1 >>> c = 1000000
2 >>> pow(c,5)+pow(c,4)+pow(c,3)+pow(c,2)+pow(c,1) < pow(c,6)
3 True
4 # 1000001000001000001000001000001000000 < 1000000000000000000000000000000000000
```

简而言之，我们总可以找到一个 $c = f(n)$ 的最高次幂项的系数 $+1$ ，以及一个 $n \rightarrow \infty$ 满足： $n > n_0$ 总有 $f(n) \leq c * g(n)$ 。

通常我们只关心**渐近时间复杂度(Asymptotic time complexity)**，也就是说，当 n 很大时：

$$O(\log(n)) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(2^n)$$

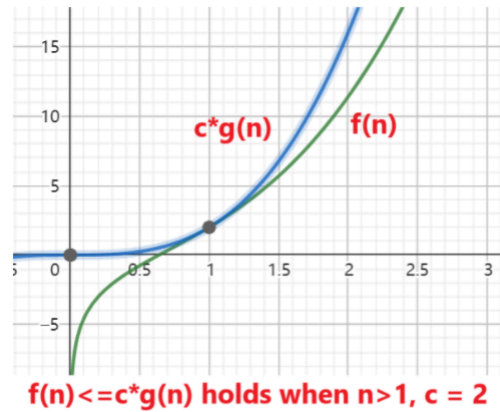
PPT习题

证明 $2n^2 + 4n$ 是 $O(n^2)$

- 因为对于所有 $n_0 > 4$, $c = 3$, $n \leq n^2$, 我们有 $2n^2 + 4n \leq 3n^2$ 对于所有 $n > n_0$
- 因此, 根据定义, $2n^2 + 4n$ 是 $O(n^2)$ 。

证明 $n^3 + n^2 \log n + n$ 是 $O(n^3)$

- 因为对于所有 $n_0 > 1, c = 2$, 我们有 $n^3 + n^2 \log(n) + n \leq 2n^3$, 对于所有 $n > n_0$
- 因此, 根据定义, $n^3 + n^2 \log n + n$ 是 $O(n^3)$ 。



这些证明展示了如何通过不等式分析，将多项式表达式归约为主导项，从而证明其渐近复杂度。

一、证明以下函数的数量级 (Prove the order of magnitude)

1. $n^3 + 3n^2 + 3$

该函数的数量级是 $O(n^3)$ 。证明如下：

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $n_0^3 + 3n_0^2 + 3 \leq 2 * n_0^3 \implies 3n_0^2 + 3 \leq n_0^3$ 。

可以取 $n_0 = 4$ 。因此, $\exists c = 2, n_0 = 4$ 使得 $n^3 + 3n^2 + 3 \leq n^3$ 对于所有 $n > n_0$ 成立。

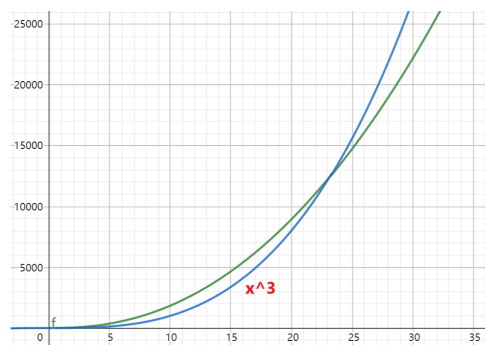
2. $4n^2 \log(n) + n^3 + 5n^2 + n$

该函数的数量级是 $O(n^3)$ 。证明如下：

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $4n_0^2 \log(n_0) + 5n_0^2 + n_0 \leq n_0^3$ 。如果以2为底, 大约取 $n_0 = 25$ 即可。

因此, $\exists c = 2, n_0 = 25$ 使得 $4n^2 \log(n) + n^3 + 5n^2 + n \leq 2 * n^3$ 对于所有 $n > n_0$ 成立



3. $2n^2 + n^2 \log(n)$

该函数的数量级是 $O(n^2 \log(n))$ 。证明如下：

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $2n_0^2 \leq n_0^2 \log(n_0) \implies n_0 \geq 4$ 。可选 $n_0 = 4$ 。

因此, $\exists c = 2, n_0 = 4$ 使得 $2n^2 + n^2 \log(n) \leq 2 * n^2 \log(n)$ 对于所有 $n > n_0$ 成立

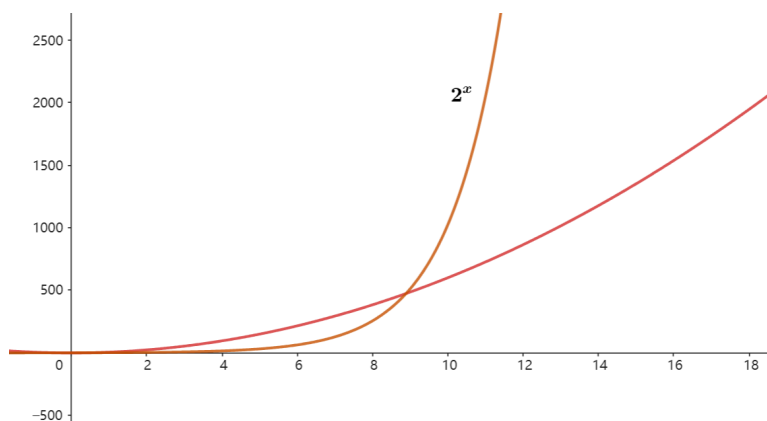
4. $6n^2 + 2^n$

该函数的数量级是 $O(2^n)$ 。证明如下：

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $6n_0^2 \leq 2^{n_0}$, 可选 $n_0 = 10$ 。

因此 $\exists c = 2, n_0 = 10$ 使得 $6n^2 + 2^n \leq 2 * 2^n$ 对于所有 $n > n_0$ 成立

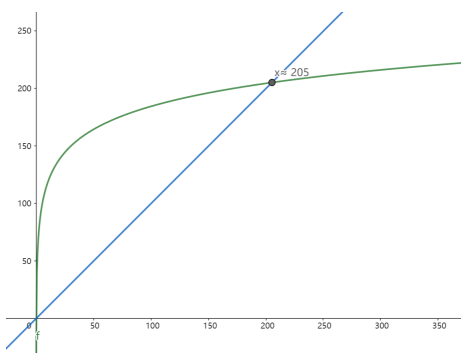


二、写出 $6n^{20} + 2^n$ 的数量级

$6n^{20} + 2^n$ 的数量级是 $O(2^n)$ 。为什么不是 n^{20} ?

不妨假设 $c = 1 + 1 = 2$ 。先求出 n_0 : $6n_0^{20} \leq 2^{n_0}$ 。如果说直接看的话, 就很难看出来结果。但是我们可以试着取对数:

$$\begin{aligned} 6n_0^{20} &\leq 2^{n_0} \\ 20 * \log(6n_0) &\leq n_0 \log(2) \\ 20 * \log(6n_0) &\leq n_0 \\ n_0 &\approx 205 \end{aligned}$$



也就是, $\exists c = 2, n_0 = 205$ 使得 $6n^{20} + 2^n \leq 2 * 2^n$ 对于所有 $n > n_0$ 成立。

■ [Week 2]

算法效率 + 搜索/排序

- ✓ 了解一些多项式时间和指数时间算法的例子
- ✓ 能够对算法进行简单的渐进分析
- ✓ 能够应用搜索/排序算法并推导其时间复杂度

[1] 线性搜索

```
1 public static int linearSearch(int[] array, int number)
2 {
3     for(int i =0;i<array.length;i++)
4         if(array[i]==number)
5             return i;        // 找到了, 返回index
6     return -1;              // 没找到index, 返回-1
7 }
```

最好情况: $O(1)$, 最坏情况, $O(n)$ 。平均而言 $O(\frac{n}{2})$ 也就是 $O(n)$ 。

[2] 二分搜索

如果说数据有序 (例如升序) , 那么我们就可以使用二分搜索的方法来减少时间复杂度。

递归 Recursive 方法:

```
1 public static int binarySearch(int[] array, int number){
2     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
3     else return binarySearch(array,number,0,array.length-1); // 开始二分搜索
4 }
5 private static int binarySearch(int[] array,int number, int left, int right){
6     if(left>right) return -1; // 如果左边大于右边, 直接返回-1。因为越界代表找不到
7     int middleValue = array[(left+right)/2]; // 定义中间值
8     if(number == middleValue) return (left+right)/2; // 如果相等就返回
9     if(number < middleValue) return binarySearch(array,number,left,(left+right)/2-1);
10    // 如果number比中间值小, 就往左边找
11    return binarySearch(array,number,(left+right)/2+1,right); // 如果number比中间值大, 就
    往右边找
11 }
```

循环方法 (更简单) :

```
1 public static int binarySearchLoop(int[] array, int number)
2 {
3     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
4     int left = 0;
5     int right = array.length-1;
6     while(left<=right)
7     {
8         int middleValue = array[(left+right)/2];
9         if(number > middleValue){ // 往右边找
10             left = (left+right)/2 +1;
11             continue;
12         }
13         if(number < middleValue){ // 往左边找
14             right = (left+right)/2-1;
15             continue;
16         }
17         if(number == middleValue)
```

```

18         return (left+right)/2;
19     }
20     return -1;
21 }

```

最好情况 $O(1)$ 。现在计算最坏情况。假设我们有 n 个数据。那么假设这个数据恰好要对比到最后一次，也就是需要比较 $\lceil \log_2(n) \rceil + 1$ 次。

如果一个数组 `arr = [0,1,2,3,4,5,6,7,8,9]`。我们想找到数字 6。

第一次： `middle = arr[(0+9)/2] = arr[4] = 4`。继续搜索下标 5~9。

第二次： `middle = arr[7] = 7`。继续搜索下标 5~6。

第三次： `middle = arr[5] = 5`。继续搜索下标 6~6。

第四次：找到了 6。

一共计算了 $\log_2(10) + 1 = 4$ 次。

因此最坏的时间复杂度是 $O(\log_2(n) + 1)$ 。平均而言就是 $O((\log_2(n) + 2)/2) = O(\frac{1}{2}\log_2(n) + 1)$ 。最后，时间复杂度是 $O(\log_2(n))$

对比而言，二分搜索更有效率！

经过测试，如果用线性搜索 400,000,000 个数据，平均每次查找需要 81.34 ms。而使用二分搜索，平均每次查询只需要 5.578×10^{-4} ms。

[3] 字符串匹配

给定一个文本 `text` (String型)和一个模式 `pattern`。要求找到这个文本 `text` 内是否含有 `pattern`。

```

1  public static boolean exist(String text, String pattern)
2  {
3      if(text==null||pattern==null|| text.isEmpty() || pattern.isEmpty()) return false;
4      if(pattern.length()>text.length()) return false;
5
6
7      for(int i = 0;i<text.length();i++)                // 第一层循环
8      {
9          for(int j = 0; j<pattern.length();j++)        // 第二层循环
10         {
11             if(pattern.charAt(j)!=text.charAt(j+i)) break;
12             if(j==pattern.length()-1) return true;
13         }
14     }
15     return false;
16 }

```


假设 `text` 的长度是 n ，`pattern` 的长度是 m 。这里嵌套了两层循环。最好的情况是 `pattern` 就在开头，也就是 $O(m)$ 时间。

最坏的情况是找到末尾才结束。也就是大约遍历了 $m \times n$ 次，即 $O(nm)$ 。总时间复杂度就是 $O(nm)$ 。

[4] 选择排序

步骤如下：

原始数组 `originalArray = {5,1,3,2,4,0}`。

找出 `originalArray = {5,1,3,2,4,0}` 最小的，得到 0，放入新数组 `sortedArray = {0}`

找出 `originalArray = {5,1,3,2,4}` 最小的，得到 1，放入新数组 `sortedArray = {0,1}`

找出 `originalArray = {5,3,2,4}` 最小的，得到 2，放入新数组 `sortedArray = {0,1,2}`

...

这样就能排序好了。

实际上，并不需要创建两个数组。只需要进行**交换操作**即可

```
1 public static void selectionSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int i = 0; i<array.length-1;i++){
4         {
5             for(int j=i+1;j<array.length;j++){
6                 {
7                     if(array[j]<array[i])
8                         swap(array,i,j);
9                 }
10            }
11        }
12    private static void swap(int[] array, int indexA, int indexB)
13        {
14            int temp = array[indexA];
15            array[indexA] = array[indexB];
16            array[indexB] = temp;
17        }
```

计算时间复杂度：

这个算法没有极端情况和最好情况。

当 $i = 0$ 的时候， j 遍历 $n-1$ 次。

当 $i = 1$ 的时候， j 遍历 $n-2$ 次。

...

因此总共需要的执行数是 $(n-1) + (n-2) + \dots + 1 = \frac{n \times (n-1)}{2}$

因此时间复杂度是 $O(n^2)$ 。

[5] 冒泡排序

冒泡排序的思路和选择排序差不多。可以看[这个视频](#)。

```
1 public static void bubbleSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int end = array.length-1; end>=0; end--){
4         {
5             for(int begin = 0; begin<end; begin++){
6                 {
7                     if(array[begin]>array[begin+1]) swap(array, begin, begin+1);
8                 }
9             }
10    }
```

同样的，它的时间复杂度仍然是 $O(n^2)$ 。

选择排序交换的是起始索引（一直往后移动）和动索引，界限是前面。而冒泡排序则是交换两个相邻索引，界限是在后面。

[6] 插入排序(Optional)

插入排序，可以看[这个视频](#)。

```
1 public static void insertSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int i = 1; i<array.length; i++){
4         {
5             int j = i-1;
6             while(j>=0 && array[j]>array[j+1])
7                 {
8                     swap(array, j, j+1);
9                     j--;
10                }
11        }
12    }
```

总比较数（最坏情况）： $1 + 2 + \dots + (n - 1) = \frac{n \times (n - 1)}{2}$ ，时间复杂度为 $O(n^2)$ 。

[7] 小结

选择排序、冒泡排序、插入排序：这三种算法在最坏情况下的时间复杂度均为 $O(n^2)$ 。

是否存在更高效的排序算法？**是的**，我们后续会学习它们。

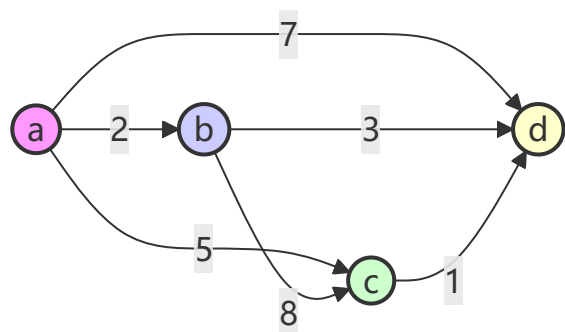
基于比较的最快排序算法的时间复杂度是多少？ $O(n \log(n))$

一些**指数时间复杂度**的算法包括：

- **旅行商问题** (Traveling Salesman Problem)
- **背包问题** (Knapsack Problem)

旅行商问题

输入：共有 n 个城市。
输出：找到从某一特定城市出发的最短路径，要求恰好访问每个城市一次，并最终返回到起点城市。
这个问题被称为**哈密顿回路**（Hamiltonian Circuit）。



路径	计算方式	总距离
a -> b -> c -> d -> a	$2 + 8 + 1 + 7$	18
a -> b -> d -> c -> a	$2 + 3 + 1 + 5$	11
a -> c -> b -> d -> a	$5 + 8 + 3 + 7$	23
a -> c -> d -> b -> a	$5 + 1 + 3 + 2$	11
a -> d -> b -> c -> a	$7 + 3 + 8 + 5$	23
a -> d -> c -> b -> a	$7 + 1 + 8 + 2$	18

但是，随着城市的增加，可能会出现指数级别的爆炸。如果城市数量为 n ，那么需要考虑的数量是 $(n - 1)!$
时间复杂度为 $O((n - 1)!)$ 。比如上图一共四个城市，总共可能的回路数量是 $(4 - 1)! = 6$

背包问题

输入：给定 n 个物品，每个物品有重量 $w_1, w_2, \dots, w_n$ 和价值 $v_1, v_2, \dots, v_n$ ，以及一个容量为 w 的背包。
输出：找到一个可以放入背包且总价值最大的物品子集。
应用：一架运输机需要将最有价值的物品集运送到一个偏远地点，同时不超出其容量限制。
如果有四个物品，也就是 $n = 4$ ，这种情况可能的数量是 $2^4 = 16$ 。时间复杂度： $O(2^n)$ ，

Q&A

假设你忘记了由5个字符组成的密码。你只记得：

- 这5个字符都是不同的。
- 这5个字符分别是 B、D、M、P、Y。

如果你想尝试所有可能的组合，总共有多少种？

5!

如果这5个字符可以是26个大写字母中的任意一个，那么总共有多少种？

26^5 (如果重复)

A_{26}^5 (如果不重复)

假设密码还包含2个数字，即总共有7个字符：

- 所有字符都是**不同的**。
- 5个字母分别是 B、D、M、P、Y。
- 数字只能是 0 或 1。

总共有多少种组合？

$C_7^2 A_2^2 A_5^5$

假设密码的形式是 `adaaada`，其中：

- `a` 表示字母，`d` 表示数字。
- 所有字符都是不同的。
- 5个字母分别是 B、D、M、P、Y。
- 数字只能是 0 或 1。

总共有多少种组合？

$A_2^2 A_5^5$

Tutorial

1. Give the trace table and the output of the following algorithm for $m = 16$.

```
1  input m
2  count = 0
3  x = 1
4  while x < m do
5      begin
6          x = x * 2
7          count = count + 1
8      end
9  output count
```

What is the output for general m that is a positive power of 2 (i.e., $m = 2, 4, 8, 16, 32, \dots$)?

$\log_2(m) = \log_2(16) = 4$

2. Rewrite the following for-loop into a while loop.

```

1 input m
2 x = 0
3 for i = 1 to m do
4     begin
5         x = x + i
6     end
7 output x

```

```

1 input m
2 x = 0
3 while m>0 do
4     begin
5         x = x + i
6         m--;
7     end
8 output x

```

3. Rewrite the above for-loop into a repeat loop.

```

1 input m
2 x = 0
3 repeat
4     x = x + i;
5     m--;
6 until m = 0

```

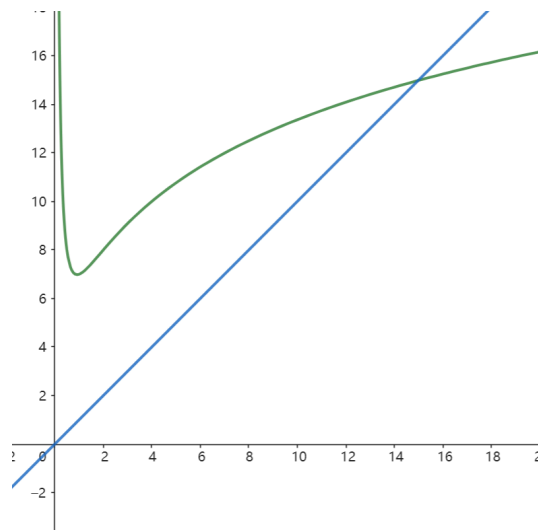
4. List the following functions from lowest order to highest order of magnitude:

将以下函数从最低阶到最高阶的量级进行排序：

$O(1)$ 、 $O(\log(n))$ 、 $O(\log^2(n))$ 、 $O(\sqrt{n})$ 、 $O(n)$ 、 $O(n\log(n))$ 、 $O(n^{\frac{3}{2}})$ 、 $O(n^2)$ 、 $O(n^2\log^4(n))$ 、 $O(n^3)$ 、 $O(n^{\frac{10}{3}})$ 、 $O(n^4)$

5. Prove that the function $f(n) = 3n^2\log n + 2n^3 + 3n^2 + 4n$ is $O(n^3)$.

令 $c = 2 + 1 = 3$ 。解不等式 $3n_0^2\log n_0 + 2n_0^3 + 3n_0^2 + 4n_0 < 3n_0^3$ 得到 $n_0 \approx 15$ 取 $n_0 = 16$
 所以 $\exists c = 3, n_0 = 16$ 使得 $f(n) \leq c \cdot g(n), g(n) = n^3$ 对于所有的 $n > n_0$ 成立



6. Consider a string T of n characters $T[0..(n-1)]$. Design and write a pseudo code of an algorithm to determine if a given character, denoted by C , appears uniquely in $T[0..(n-1)]$, i.e., whether the character C appears exactly once in T .

For example, if T is "Hello, how are you?" and C is **H**, then the algorithm should report "Yes, the character appears uniquely.". However, if C is **e**, then the algorithm should report "No, the character does not appear uniquely."

考虑一个由 n 个字符组成的字符串 T ，其索引范围为 $T[0..(n-1)]$ 。设计并编写一个伪代码算法，用于判断给定的字符 C 是否在 $T[0..(n-1)]$ 中唯一出现，即字符 C 是否在 T 中恰好出现一次。

例如，如果 T 是 "Hello, how are you?"，且字符 C 是 H，那么算法应报告 "Yes, the character appears uniquely." ("是的，该字符唯一出现。")。然而，如果 C 是 e，那么算法应报告 "No, the character does not appear uniquely." ("不，该字符并未唯一出现。")。

```
1 public static void isAppearAtOnce(String text, String pattern){
2     if(countAppearTime(text,pattern)>1)
3         System.out.println("No, the character does not appear uniquely.");
4     else
5         System.out.println("Yes, the character appears uniquely.");
6 }
7 private static int countAppearTime(String text,String pattern){
8     if(text==null||pattern==null|| text.isEmpty() || pattern.isEmpty()) return 0;
9     if(pattern.length()>text.length()) return 0;
10    int count = 0;
11    for(int i = 0;i<text.length();i++)
12    {
13        for(int j = 0; j<pattern.length();j++)
14        {
15            if(pattern.charAt(j)!=text.charAt(j+i)) break;
16            if(j==pattern.length()-1) count++;
17        }
18    }
19    return count;
20 }
```

■ [Week 3]

分治 (Divide and Conquer) 算法

本周有两个Lecture无Tutorial，均包含在本节

学习目标

- 理解分治法的工作原理，并能够通过求解递推关系来分析分治法的时间复杂度。
- 查看分治法的示例。

[1] 关于分治

分治法是最著名的算法设计技术之一。

核心理念：

1. 将一个问题的实例划分为多个相同问题的较小实例，理想情况下这些实例的规模大致相同。

2. 递归地解决这些较小的实例。
3. 将较小实例的解合并，得到原问题的解。

回顾二分搜索

回顾我们之前学过的二分搜索/查找：

输入： 一个包含 n 个已排序数字的序列 $a_0, a_1, \dots, a_{n-1}$ ； 以及一个数字 X 。

算法思想：

1. 将 X 与中间位置的数字进行比较。
2. 根据 X 是小于还是大于中间数字，将搜索范围缩小到前半部分或后半部分。
3. 将需要搜索的数字数量减少一半。

我们用了 递归 Recursive 的方法，也用了一般while循环的方法，都可以实现。

时间复杂度

设 $T(n)$ 表示二分查找算法在 n 个数字上的时间复杂度

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\frac{n}{2}) + 1 & \text{otherwise} \end{cases}$$

我们称这个公式为一个递推关系

二分搜索的时间复杂度

假设 $T(n) \leq 2 \log n$

递推关系为：

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\frac{n}{2}) + 1 & \text{otherwise} \end{cases}$$

验证基本情况：

猜： $T(n) \leq 2 \log n$

$$T(n) \leq 2 \log n$$

其中L.H.S (Left Hand Side, 左边) 为 $T(n)$

R.H.S 为 $2 \log n$

代入法 (Substitution method)

当 $n = 1$ 时，假设不成立！

$$\text{左边 (L.H.S)} = T(1) = 1$$

$$\text{右边 (R.H.S)} = c \log 1 = 0$$

显然, $L.H.S > R.H.S$

然而, 当 $n = 2$ 时:

$$\text{左边 (L.H.S)} = T(2) = T(1) + 1 = 2$$

$$\text{右边 (R.H.S)} = 2 \log 2 = 2$$

此时, $L.H.S \leq R.H.S$ 成立。

具体而言怎么操作呢?

$$T(n) = T\left(\frac{n}{2}\right) + 1 \leq 2 \log\left(\frac{n}{2}\right) + 1 = 2(\log n - 1) + 1 = 2 \log n - 1$$

↑ 带入 $T(n) = 2 \log n$

$$T(n) \leq 2 \log n - 1$$

相当于我们把recursive的部分当作 $\log$ 处理。

那么这个时间复杂度就是 $O(2 \log n - 1) = O(\log n)$

练习

Q1

求如下算法时间复杂度

$$T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + 1 & \text{otherwise} \end{cases}$$

在这里, 我们先试着枚举一下:

$$T(1) = 1$$

$$T(2) = 2T(1) + 1 = 3$$

$$T(3) = 2T(1) + 1 = 3 \text{ (Rounding, } 3 \div 2 = 1)$$

$$T(4) = 2T(2) + 1 = 7$$

$$T(5) = 2T(2) + 1 = 7 \text{ (Rounding)}$$

$$T(6) = 2T(3) + 1 = 7$$

$$T(7) = 2T(3) + 1 = 7 \text{ (Rounding)}$$

$$T(8) = 2T(4) + 1 = 15$$

可以看到, 数字 2, 4, 6, 8 的规律是: $2n - 1$ 。那么就假设这个时间复杂度满足

$$T(n) \leq 2n - 1$$

证明如下:

$$T(n) = 2T\left(\frac{n}{2}\right) + 1 \leq 2\left(2\left(\frac{n}{2}\right) - 1\right)$$

↑这里要证明 $T(n) \leq 2n - 1$ ，那么就直接带入 $T(n) = 2n - 1$
 如果最后化简出来的能满足 $\leq 2n - 1$ 就可以证明了
 $= 2n - 1$

显然是满足的

因此时间复杂度 $O(2n - 1) = O(n)$

PPT介绍了猜测的方法，但是我认为这样比较难猜

一般而言，我们可以采用以下方法：考虑 $n > 1$ 的情况

$$T(n) = 2T\left(\frac{n}{2}\right) + 1 \quad \text{接下来展开 } T\left(\frac{n}{2}\right)。 \text{展开第1次}$$

$$= 2\left(2T\left(\frac{n}{4}\right) + 1\right) + 1 = 4T\left(\frac{n}{4}\right) + 3 \quad \text{展开 } T\left(\frac{n}{4}\right)。 \text{展开第2次}$$

$$= 4\left(2T\left(\frac{n}{8}\right) + 1\right) + 3 = 8T\left(\frac{n}{8}\right) + 7 \quad \text{展开第3次}$$

$$= 16T\left(\frac{n}{16}\right) + 15 \quad \text{展开第4次}$$

$$= \dots \quad \text{因为是2的指数倍，所以一共展开了 } \log_2 n \text{ 次，因此}$$

$$= 2^{\log_2 n} T(1) + 2^{\log_2 n} - 1 = nT(1) + n - 1 = 2n - 1$$

因此时间复杂度 $O(2n - 1) = O(n)$

Q2

求如下算法时间复杂度

$$T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + n & \text{otherwise} \end{cases}$$

$$\begin{aligned}
T(n) &= 2T\left(\frac{n}{2}\right) + n \\
&= 4T\left(\frac{n}{4}\right) + 2n \\
&= 8T\left(\frac{n}{8}\right) + 3n \\
&= 16T\left(\frac{n}{16}\right) + 4n \\
&= \dots \\
&= 2^{\log_2 n} T(1) + n \log_2 n \\
&= n + n \log_2 n
\end{aligned}$$

时间复杂度： $O(n + n \log n) = O(n \log n)$

使用猜测的方法也有一些技巧，如下：

根据递推关系，我们可以猜测其数量级：

- $T(n) = T\left(\frac{n}{2}\right) + 1$, $T(n)$ 是 $O(\log n)$
- $T(n) = 2T\left(\frac{n}{2}\right) + 1$, $T(n)$ 是 $O(n)$
- $T(n) = 2T\left(\frac{n}{2}\right) + n$, $T(n)$ 是 $O(n \log n)$

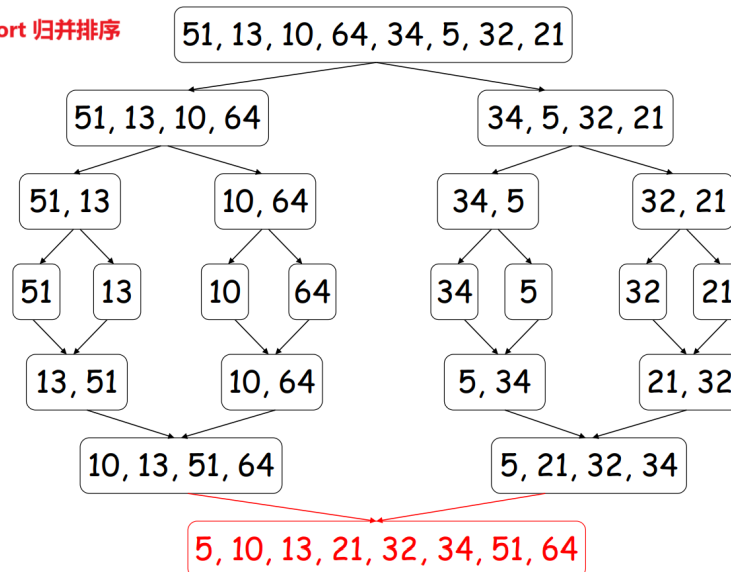
[2] 归并排序

Merge Sort

归并排序（Merge Sort）是一种基于分治法（Divide and Conquer）的排序算法。

其主要思想是将待排序的序列不断分成两半，递归地对每一半进行排序，然后将两个已排序的子序列合并成一个完整的有序序列。归并排序的时间复杂度为 $O(n \log n)$ ，是一种稳定的排序算法。

Merge Sort 归并排序



```

1 public static void mergeSort(int[] arr) {
2     if (arr == null || arr.length < 2) return;
3     mergeSort(arr, 0, arr.length - 1);
4 }
5
6 private static void mergeSort(int[] array, int left, int right) {
7     // 这里左右都要分，分到最小单元之后merge。
8     if (left < right) {
9         int middle = left + (right - left) / 2;
10        mergeSort(array, left, middle);
11        mergeSort(array, middle + 1, right);
12        merge(array, left, middle, right);
13    }
14 }
15
16 private static void merge(int[] array, int left, int middle, int right) {
17     // 第一个数组: left 到 middle
18     // 第二个数组: middle+1 到 right
19     // 现在选出两个数组最小的放在temp中
20     int[] temp = new int[right - left + 1];
21     int indexOfTemp = -1;
22     int firstArrayIndex = left;
23     int secondArrayIndex = middle + 1;
24     while (firstArrayIndex <= middle && secondArrayIndex <= right)
25         if (array[firstArrayIndex] < array[secondArrayIndex])
26             temp[++indexOfTemp] = array[firstArrayIndex++];
27         else
28             temp[++indexOfTemp] = array[secondArrayIndex++];
29
30     // 复制剩余数组到 temp
31     if (firstArrayIndex == middle + 1) // 如果first到末尾了
32         while (secondArrayIndex <= right) temp[++indexOfTemp] =
33         array[secondArrayIndex++];
34     else while (firstArrayIndex <= middle) temp[++indexOfTemp] =
35     array[firstArrayIndex++];

```

```

34
35      // temp替换原来数组
36      indexOfTemp = -1;
37      while (++indexOfTemp < temp.length)
38          array[left + indexOfTemp] = temp[indexOfTemp];
39  }
40
41  public static void main(String[] args) {
42      int[] arr = {51, 13, 10, 64, 34, 5, 32, 21};
43      mergeSort(arr);
44      System.out.println(Arrays.toString(arr));
45  }

```

分析时间复杂度：

对于含有 n 个元素的数组，先拆分为 n 个数组，然后：

归并1次：得到 $\frac{n}{2}$ 个数组

归并2次：得到 $\frac{n}{4}$ 个数组

归并3次：得到 $\frac{n}{8}$ 个数组

...

一共需要归并 $\log_2 n$ 次。然而每次归并都需要遍历 n 次，因此时间复杂度是 $O(n \log n)$

归并排序的时间复杂度递推公式 $T(n) = 2T\left(\frac{n}{2}\right) + n$ 的推导如下：

1. Divide (分) :

将长度为 n 的序列分成两部分，每部分的长度为 $\frac{n}{2}$ ，即需要递归地对两个子序列进行排序。因此， $T(n)$ 包含了两次对长度为 $\frac{n}{2}$ 的序列的排序，即 $2T\left(\frac{n}{2}\right)$ 。

2. Conquer (治) :

合并两个已排序的子序列时，需要线性时间 $O(n)$ ，因为每个元素最多被比较和移动一次。因此， $T(n)$ 还包含一个线性项 n 。

3. Base Case (基本情况) :

当 $n = 1$ 时，不需要任何操作，因此 $T(1) = 1$ 。

综上所述，归并排序的时间复杂度递推公式为：

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{otherwise} \end{cases}$$

[3] 关于图

学习目标

✓ 能够区分什么是无向图 (Undirected Graph)，什么是有向图 (Directed Graph)

掌握如何使用矩阵 (Matrix) 和邻接表 (List) 表示图

✓ 理解欧拉路径 (Euler Path) / 欧拉回路 (Euler Circuit)，并能够判断在一个无向图中是否存在这样的路径 / 回路

✓ 能够应用广度优先搜索 (BFS) 和深度优先搜索 (DFS) 遍历图

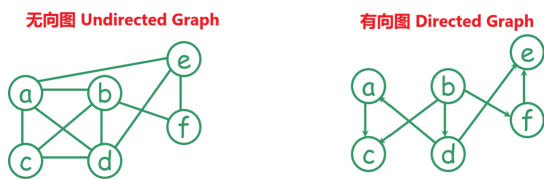
✓ 能够描述什么是树 (Tree)

图论：一个历史悠久但具有许多现代应用的主题。

起源于 18 世纪。

无向图 $G = (V, E)$ 由一组顶点 V 和一组边 E 组成。每条边是一个顶点的无序对。（例如， $\{b, c\}$ 和 $\{c, b\}$ 表示同一条边。）

有向图 $G = (V, E)$ 由一组顶点 V 和一组边 E 组成。每条边是一个顶点的有序对。（例如， (b, c) 表示一条从 b 到 c 的边。）



图的应用

在计算机科学中，图通常用于建模：

- 计算机网络，
- 进程之间的优先关系，
- 下棋的状态空间（人工智能应用），
- 资源冲突等。

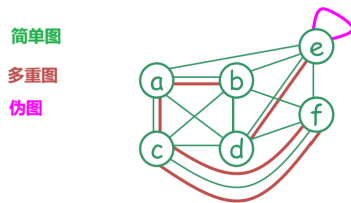
在其他学科中，图也用于对对象的结构进行建模。例如：

- 生物学——进化关系，
- 化学——分子结构。

无向图

无向图分为以下几种：

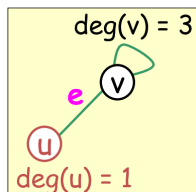
- **简单图 (Simple Graph)**：两个顶点之间最多有一条边，不存在自环（即从一个顶点到自身的边）。
- **多重图 (Multigraph)**：允许两个顶点之间存在多条边。
- **伪图 (Pseudograph)**：允许存在自环。



提示：一个无向图 $G = (V, E)$ 由一组顶点 V 和一组边 E 组成。每条边是一个顶点的无序对。

在无向图 G 中，假设 $e = \{u, v\}$ 是 G 的一条边：

- u 和 v 被称为**相邻 (adjacent)**，并且互为**邻居 (neighbors)**。
- u 和 v 被称为边 e 的**端点 (endpoints)**。
- 边 e 被称为**关联 (incident)** 于顶点 u 和 v 。
- 边 e 被称为**连接 (connect)** 顶点 u 和 v 。
- 顶点 v 的**度 (degree)**，记作 $\deg(v)$ ，是与其关联的边的数量（自环对度的贡献为 2 次）。



无向图的表示

无向图可以通过以下方式表示：

- **邻接矩阵 (Adjacency Matrix)**
- **邻接表 (Adjacency List)**
- **关联矩阵 (Incidence Matrix)**
- **关联表 (Incidence List)**

邻接矩阵和**邻接表**记录顶点之间的邻接关系，即一个顶点与哪些其他顶点相邻。

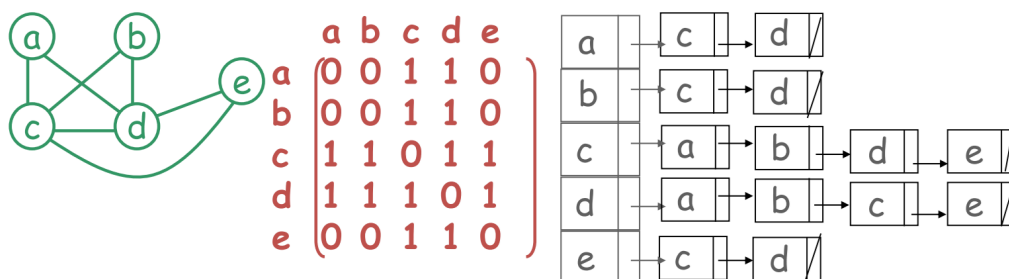
关联矩阵和**关联表**记录边与顶点之间的关联关系，即一条边关联于哪两个顶点。

邻接矩阵 / 邻接表

对于一个具有 n 个顶点的**简单无向图**，其邻接矩阵 M 定义如下：

- M 是一个 $n \times n$ 的矩阵
- 如果顶点 i 和顶点 j 相邻，则 $M(i, j) = 1$ ，否则 $M(i, j) = 0$

邻接表：每个顶点都有一个列表，记录与之相邻的所有顶点。

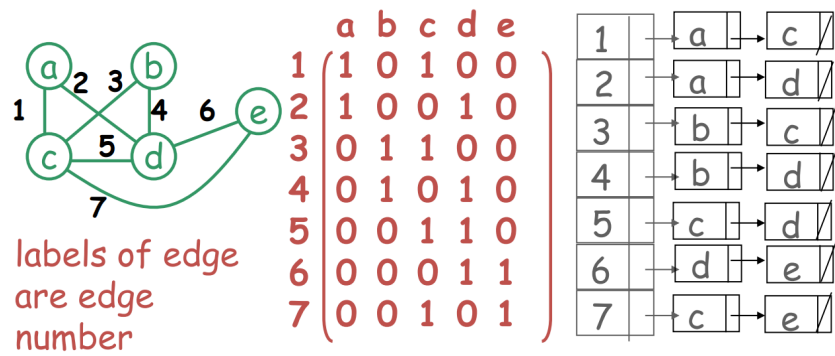


关联矩阵 / 关联表

对于一个具有 n 个顶点和 m 条边的**简单无向图**，其关联矩阵 M 定义如下：

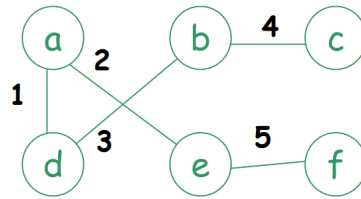
- M 是一个 $m \times n$ 的矩阵
- 如果边 i 和顶点 j 相关联，则 $M(i, j) = 1$ ，否则 $M(i, j) = 0$

关联表：每条边都有一个列表，记录与之关联的所有顶点。



练习

给定以下图，请写出其邻接矩阵和关联矩阵。



邻接矩阵：

	a	b	c	d	e	f
a				1	1	
b			1	1		
c		1				
d	1	1				
e	1					1
f					1	

空白处填0

关联矩阵：

	a	b	c	d	e	f
1	1			1		
2	1				1	
3		1		1		
4		1	1			
5					1	1

欧拉回路、欧拉路径

□ 在无向图中，从顶点 u 到顶点 v 的路径 Path 是一系列边 $e_1 = \{u, x_1\}, e_2 = \{x_1, x_2\}, \dots, e_n = \{x_{n-1}, v\}$ ，其中 $n \geq 1$ 。

□ 该路径的长度为 n 。

□ 注意，从 u 到 v 的路径意味着从 v 到 u 的路径。

□ 如果 $u = v$ ，则该路径称为回路（环）Circuit。

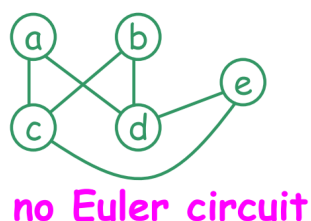
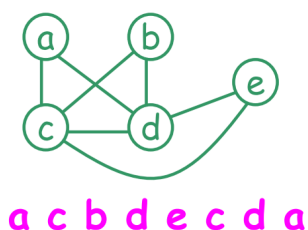
欧拉回路 Euler Circuit

一个简单的回路最多访问每条边一次。

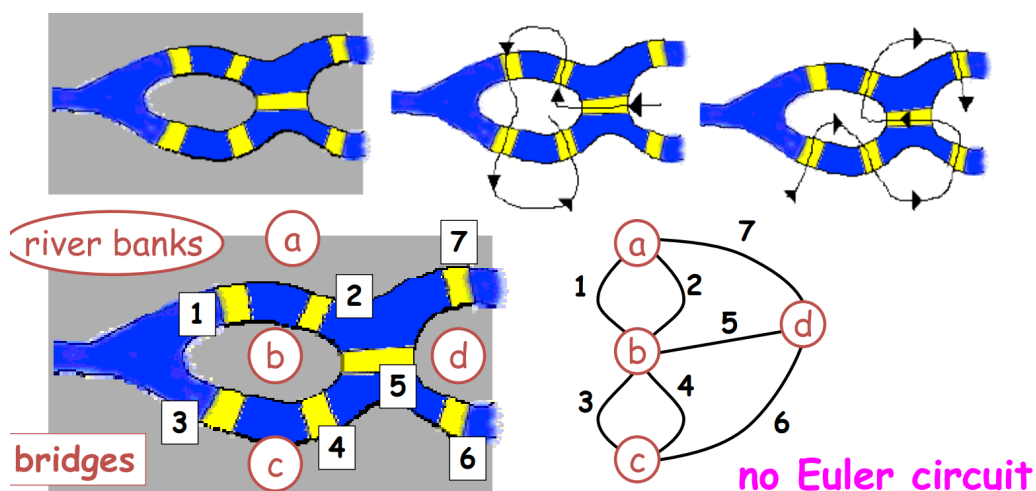
在图 G 中，欧拉回路是指访问图 G 中每条边恰好一次的回路。

（注意：顶点可以被重复访问。）

每个图都有欧拉回路吗？显然不是



历史背景：在德国柯尼斯堡，一条河流穿过城市，并建造了七座桥梁。市民们想知道是否有一种方式可以让人们环游城市，同时恰好每座桥只经过一次。可惜，并没有这种路径



靠直接判断肯定行不通，但是有什么办法能判断这个图有无欧拉回路呢？

一个无向图 G 被称为**连通的**(connected)，如果每一对顶点之间都存在一条路径。

不连通固然没有欧拉回路。但是即使图是连通的，也可能不存在欧拉回路。

充要条件 Necessary and Sufficient Condition

设 G 为连通图。 引理： G 包含一条欧拉回路当且仅当每个顶点的度数都是偶数

欧拉路径 Eula Path

设 G 为一个无向图。 欧拉路径是一条访问 G 中每一条边恰好一次的路径。

一个无向图包含一条欧拉路径当且仅当它是连通的，并且恰好有两个顶点的度数是奇数。

给定一个连通的无向图

	存在欧拉回路?	存在欧拉路径?
所有顶点的度数均为偶数	是	是
恰好两个顶点的度数为奇数	否	是
多于两个顶点的度数为奇数	否	否

[4] 哈密顿回路/路径

Hamiltonian circuit / path

设 G 为一个无向图。

哈密顿回路（路径）是指一条回路（路径），它包含图 G 中的每个顶点恰好一次。

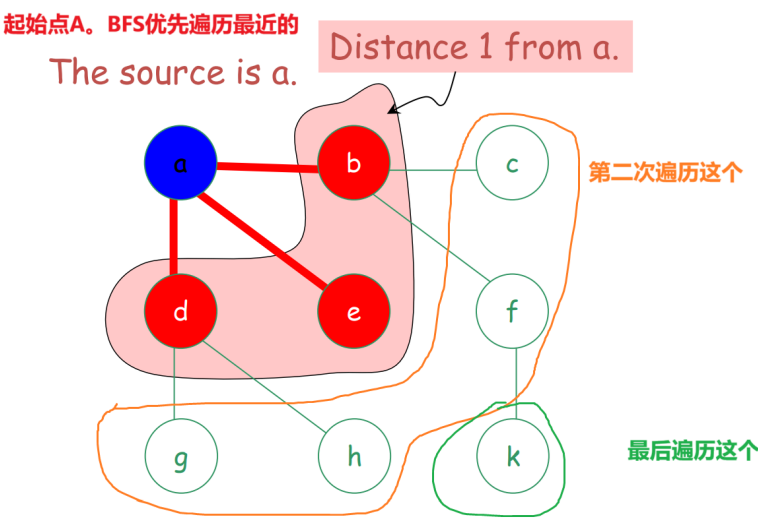
需要注意的是，哈密顿回路或路径不一定遍历所有边。

与欧拉回路/路径的情况不同，确定一个图是否包含哈密顿回路（路径）是一个非常困难的问题。（NP 难问题）

[5] 广度优先搜索

BFS - Breadth First Search

在探索距离为 $k + 1$ 的任何顶点之前，先探索所有距离起点 s 为 k 的顶点。



```

1 // 伪代码
2 取消所有点标记
3 选择一个待遍历点 s
4 标记 s
5 把 s 放入 L 中
6 while L is nonempty do
7   begin
8     remove a vertex v from front of L
9     visit v
10    for each unmarked neighbor w of v do
11      mark w and insert w into tail of list L
12  end

```

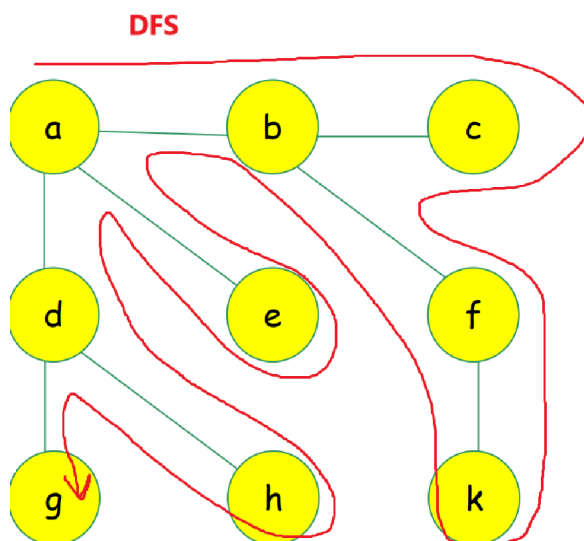
[6] 深度优先搜索

DFS - Depth First Search

深度优先搜索 (DFS)

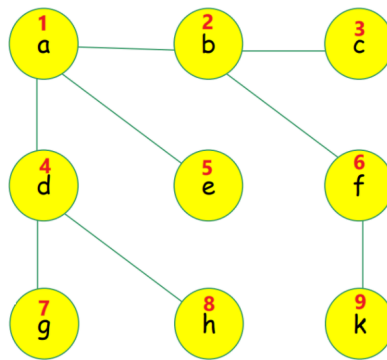
深度优先搜索是另一种探索图的策略；它尽可能在图中“深入”搜索。

- 从最近发现的顶点 v 出发，探索其尚未被探索的边。
- 当顶点 v 的所有边都被探索完毕后，搜索会“回溯”到发现 v 的顶点，继续探索其未探索的边。



[7] 实现BFS和DFS

非PPT内容



创建图

```

1 public class Graph {
2     private boolean[][] graph; // 这个数组是邻接矩阵, graph[a][b]代表ab点是否联通
3     private boolean[] flag; // 这个数组代表这个点是否被标记
4     private int size;
5     public Graph(int size) {
6         this.size = size+1;
7         createGraph();
8     }
9     private void createGraph() {
10         graph = new boolean[size][size];
11         flag = new boolean[size];
12     }
13     public void connect(int pointA, int pointB) { // 链接两个点
14         graph[pointA][pointB] = true;
15         graph[pointB][pointA] = true;
16     }
17     public boolean isConnected(int pointA, int pointB) { // 检查两个点是否链接
18         return graph[pointA][pointB];
19     }
20     public ArrayList<Integer> getAdjacentPoints(int pointA) { // 返回A的所有邻接点
21         ArrayList<Integer> adjacentPoints = new ArrayList<>();
22         for (int i = 0; i < size; i++)
23             if (graph[pointA][i])
24                 adjacentPoints.add(i);
25         return adjacentPoints;
26     }
27     public void flagThisPoint(int pointA) { // 标记这个点
28         flag[pointA] = true;
29     }
30     public void unFlagThisPoint(int pointA) { // 取消标记这个点
31         flag[pointA] = false;
32     }
33     public boolean isFlagged(int pointA) { // 返回boolean: 这个点是否被标记
34         return flag[pointA];
35     }
36     public ArrayList<Integer> getUnFlaggedPoints(int point) {
37         ArrayList<Integer> unFlaggedPoints = new ArrayList<>();
38         for (int i = 0; i < size; i++)

```

```

39     {
40         if(isConnected(point, i)&& !isFlagged(i))
41             unFlaggedPoints.add(i);
42     }
43     return unFlaggedPoints;
44 }
45 }

```

BFS

```

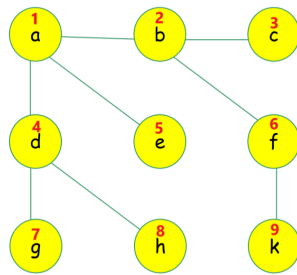
1  public class BFS {
2      public static void main(String[] args) {
3          // 创建大小为9个点的图
4          Graph graph = new Graph(9);
5          // 连接点
6          graph.connect(1,2);
7          graph.connect(2,3);
8          graph.connect(2,6);
9          graph.connect(6,9);
10         graph.connect(1,5);
11         graph.connect(1,4);
12         graph.connect(4,8);
13         graph.connect(4,7);
14
15         ArrayList<Integer> bfs = new ArrayList<>(); // 记录BFS的路径
16         ArrayList<Integer> temp = new ArrayList<>(); // 记录待遍历点，相当于缓存
17         temp.add(1);
18         graph.flagThisPoint(0); // 标记待遍历点
19         while(!temp.isEmpty())
20         {
21             int vertex = temp.removeFirst(); // 删除first点，同时赋值到vertex中
22             if(!graph.isFlagged(vertex)) // 如果这个点没有被标记过
23             {
24                 bfs.add(vertex);
25                 graph.flagThisPoint(vertex);
26             }
27             temp.addAll(graph.getUnFlaggedPoints(vertex));
28         }
29         System.out.println(Arrays.toString(bfs.toArray()));
30     }
31 }

```

输入如下：

```
1 | [1, 2, 4, 5, 3, 6, 7, 8, 9]
```

正好是广度优先搜索的顺序



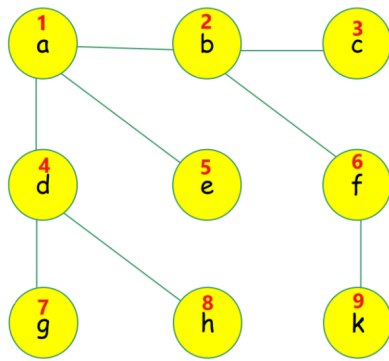
DFS

```
1 public class DFS{
2     // 创建大小为9个点的图
3     private static Graph graph = new Graph(9);
4     private static ArrayList<Integer> path = new ArrayList();
5     public static void main(String[] args) {
6         graph.connect(1,2);
7         graph.connect(2,3);
8         graph.connect(2,6);
9         graph.connect(6,9);
10        graph.connect(1,5);
11        graph.connect(1,4);
12        graph.connect(4,8);
13        graph.connect(4,7);
14        path.add(1); // 添加待遍历点
15        graph.flagThisPoint(1); // 标记待遍历点
16        dfs(1); // 开始深搜
17        System.out.println(path);
18    }
19
20    private static void dfs(int vertex) {
21        graph.getUnFlaggedPoints(vertex).forEach(point ->{
22            if(!graph.isFlagged(point)){
23                path.add(point);
24                graph.flagThisPoint(point);
25                dfs(point);
26            }
27        });
28    }
29 }
```

输出如下：

```
1 | [1, 2, 3, 6, 9, 4, 7, 8, 5]
```

确实是深度搜索：



当然换成这个例子，代码也是正确的输出

